

CS50 Cybersecurity

Lectures 0 & 1 · Securing Accounts + Securing Data

LECTURE 0

Securing Accounts

What is security?

- Security = making sure only the right person can access something
- **Authentication** = proving you are who you say you are
- Two pieces combine to prove identity: **username** (who you are) + **password** (proof it's really you)
- The whole point: only you should know both

Easy way to remember it

A lock on a door. The username is your name on the door — anyone can see it. The password is the key — only you should have it. Both are needed to get in.

Why longer passwords are harder to crack

- Hackers use **brute-force attacks** — trying every possible combination until one works
- 4-digit PIN (numbers only) = $10 \times 10 \times 10 \times 10 = 10,000$ possibilities
- 4 letters (upper + lower) = $52 \times 52 \times 52 \times 52 = 7$ million+ possibilities
- 4 characters (letters + numbers + symbols) = **78 million+ possibilities**
- Every extra character *multiplies* the possibilities — not adds them

The goal isn't to make cracking impossible — it's to make it take so long that attackers give up. This is the core idea behind all password security.

The core tradeoff: security vs usability

More secure → harder to use

Easier to use → less secure

Every security decision is a balance between these two.

NIST — the rulebook for good passwords

- **NIST** = National Institute of Standards and Technology. They write official security guidelines.
- Passwords should be **at least 8 characters**
- Systems should allow up to **64 characters** — long passphrases are encouraged
- Block passwords that appear on **breach lists** or are obvious (e.g. 'password123')
- **Don't force periodic changes** — this leads to worse passwords, not better ones
- **Lock accounts** after too many failed attempts

Two-Factor Authentication (2FA) / Multi-Factor Authentication (MFA)

The idea: even if someone steals your password, they still can't get in without a second proof. There are three categories of proof:

Knowledge	Something only you know — your password, PIN, or security answer
Possession	Something only you have — a phone, a hardware key, an OTP device
Inherence	Something only you are — fingerprint, face scan, biometrics

One-Time Passwords (OTP)

- A code that works **once only**, then expires — so even if someone sees it, they can't reuse it
- **Text message OTPs** — convenient but weak. Vulnerable to SIM swapping (attacker clones your SIM)
- **Authenticator app OTPs** — much stronger. Generated on your device, never transmitted by SMS

SIM swapping explained

An attacker calls your phone carrier, pretends to be you, and transfers your number to their SIM. Now all your SMS codes go to them. This is why app-based OTPs are safer than text messages.

Common attacks — how accounts get compromised

Brute-force attack	Trying every possible password combination until one works. Defeated by long, complex passwords.
Dictionary attack	Trying a list of common passwords instead of every combo. Faster than brute-force. Defeated by avoiding obvious passwords.
Keylogging	Malware secretly records everything you type — passwords included. Defeated by keeping devices clean of malware.
Credential stuffing	Using a leaked username/password list from one site to try to log into other sites. Defeated by using a unique password per site.
Social engineering	Tricking you into giving up credentials — not a technical attack, a people attack. Defeated by awareness.
Phishing	Fake login page that looks real. You type your password — it goes straight to the attacker. Defeated by checking URLs carefully.
Machine-in-the-Middle	Attacker intercepts data travelling between you and a server. Defeated by encrypted connections (HTTPS).

How phishing connects to social engineering

Phishing is just social engineering done digitally. Instead of a person pretending to be your bank, it's a webpage pretending to be your bank. Same trick, different medium.

How to protect yourself

Single Sign-On (SSO)

- Use your Google or Apple account to log into other services
- Fewer passwords to manage. One very secure account protects many others
- Risk: if your main account gets hacked, everything connected to it is exposed

Password Managers

- Software that generates and stores a unique complex password for every site
- You only remember **one master password** — the manager handles the rest
- Bonus: most will detect fake phishing sites and refuse to autofill
- Risk: all your eggs in one basket — protect that master password very well

Passkeys (emerging technology)

- No password at all — your device generates a **public key** and a **private key**
- The website holds the public key. Your device holds the private key. They match to prove it's you
- You never type anything — nothing to steal, nothing to phish

We'll understand exactly how this works in Lecture 1 when we cover cryptography

Quick-reference summary

- ★ Authentication = proving identity with username + password
- ★ Longer passwords = exponentially harder to brute-force
- ★ NIST: 8+ chars minimum, block breached passwords, lock after failed attempts
- ★ MFA categories: knowledge (password), possession (device), inherence (biometrics)
- ★ OTPs from apps > OTPs by SMS — SIM swapping is a real attack
- ★ Credential stuffing = reusing stolen passwords across sites — use unique passwords
- ★ Phishing = fake login pages. Never click email links — go directly to the URL
- ★ SSO and password managers both reduce the number of passwords you manage
- ★ Passkeys use public/private key cryptography — no password to steal

LECTURE 1

Securing Data

The problem — how do websites store your password?

- The naive approach: store passwords as plain text in a file
- The risk: if that file is ever stolen, every password is immediately readable
- The fix: never store the actual password — store a **hashed** version of it

Hashing — turning passwords into scrambled fingerprints

- A **hash function** takes your password and outputs a fixed-length string of gibberish
- The same input *always* produces the same output — but you can't reverse it
- The website stores the **hash**, not your password
- When you log in: your password is hashed again and compared to the stored hash

Easy way to remember it

Like a meat grinder. You put a steak in — you get mince out. You can't put the mince back in and get the steak. The website only ever sees the mince.

Remaining risks even with hashing:

- **Dictionary attack** — hash a list of common passwords and compare results
- **Rainbow tables** — pre-computed tables of billions of hashes ready to look up
- **Same password = same hash** — if two users use 'password123', their hashes are identical

Salting — fixing the 'same hash' problem

- A **salt** is a random value added to your password before it's hashed
- Even if two users have the same password, their salts are different — so their hashes are different
- This defeats rainbow tables entirely — attackers would need a separate table for every possible salt

User A: "password" + salt "xK9m" → hash("passwordxK9m") → **a8f3...**

User B: "password" + salt "p2Lq" → hash("passwordp2Lq") → **7c1e...**

NIST recommends passwords be both hashed AND salted. Both, not just one.

Cryptography — the science of hiding data

→ **Cryptography** = securely transmitting data so only the intended recipient can read it

→ Two core concepts: **codes** (swapping words for other words) and **ciphers** (scrambling the actual text)

Encryption	Converting readable text (plaintext) into unreadable scrambled text (ciphertext)
Decryption	Reversing the process — converting ciphertext back into readable plaintext
Key	The secret value used to encrypt or decrypt. Without the key, you can't read the data

Secret-key cryptography (symmetric)

→ Both the sender and the receiver have the **same key**

→ Encrypt with the key → send the ciphertext → decrypt with the same key

→ Problem: how do you share the key safely in the first place? If someone intercepts it, everything is compromised

Public-key cryptography (asymmetric) — solving the key problem

→ Each person has **two keys**: a **public key** (shared with anyone) and a **private key** (never shared)

→ Anyone can encrypt a message using your public key

→ Only your private key can decrypt it — and only you have that

Easy way to remember it

Think of a padlock and key. You hand out open padlocks to everyone (public key). Anyone can lock a box shut with your padlock. But only you have the key to open it (private key).

→ **RSA** is the most widely used standard that implements this

→ **Diffie-Hellman** is an alternative — it lets two parties establish a shared secret without ever sending it

Digital signatures — proving a message is really from you

→ Uses your **private key** to sign a document — only you could have signed it

→ Anyone with your **public key** can verify the signature is genuine

How it works step by step:

→ Step 1: the message is run through a hash function → produces a hash

→ Step 2: the hash is encrypted with your **private key** → this is the digital signature

→ Step 3: recipient hashes the message themselves and decrypts your signature with your **public key**

→ Step 4: if the two hashes match → the message is genuine and untampered

This is how passkeys (from Lecture 0) actually work under the hood — now it makes sense.

Encryption in transit — protecting data while it travels

→ Data moving across a network can be intercepted by anyone in the middle

→ **End-to-end encryption (E2EE)**: data is encrypted on your device, decrypted only on the receiver's device

- No one in between — not even the service provider — can read it
- HTTPS in your browser = the connection between you and the website is encrypted

Deleting files — it's not what you think

- When you 'delete' a file, most operating systems just **forget where it is** — the data is still there
- New files eventually overwrite that space, but there's no guarantee of timing
- **Secure deletion** = overwriting the file's location with random 1s and 0s before 'forgetting' it

Full-disk encryption

- Encrypts **everything** on your drive — even if someone steals your laptop, they get unusable data
- Requires your password or biometric to unlock it on boot
- Downside: if you forget the password and lose recovery access, your data is gone forever
- Dark side: **ransomware** uses the same technique — attackers encrypt your drive and demand payment

Quantum computing — the future threat

- Quantum computers can process certain problems **exponentially faster** than normal computers
- This could break current encryption standards — what takes years today might take hours
- The cybersecurity field is already working on **quantum-resistant** encryption algorithms

This doesn't affect you today — but it's why encryption standards keep evolving.

Quick-reference summary

- ★ Never store passwords as plain text — always hash them
- ★ Hashing = one-way transformation. Can't reverse it to find the original password
- ★ Salting = adding a random value before hashing — defeats rainbow tables
- ★ NIST: passwords must be hashed AND salted
- ★ Symmetric encryption = both sides share one key. Problem: how do you share the key safely?
- ★ Asymmetric encryption = public key (anyone can encrypt) + private key (only you decrypt)
- ★ Digital signatures use your private key to sign — verified with your public key
- ★ End-to-end encryption = no one in the middle can read it, not even the provider
- ★ 'Deleting' a file doesn't erase it — secure deletion overwrites with random data
- ★ Full-disk encryption protects your device if stolen. Ransomware abuses the same tech
- ★ Quantum computing is a future threat to current encryption — the field is preparing

How Lectures 0 & 1 connect

Lecture 0 introduced the problem: accounts get compromised through weak passwords and human attacks. Lecture 1 explains the underlying technology that makes protection actually work.

Passkeys (L0)	Mentioned as an emerging login method. Lecture 1 explains the public/private key cryptography that makes passkeys possible.
Password managers (L0)	Store hashed and encrypted versions of your passwords. Lecture 1 explains how that encryption works.
Phishing (L0)	Tricks you into giving your password. Lecture 1 explains why encrypted connections (HTTPS) make it harder but don't fully stop it.

Hashing (L1)

The backend version of the same goal as long passwords in L0 — making it very expensive for attackers to reverse-engineer credentials.

Digital signatures (L1)

The same public/private key system from passkeys. Proves identity without ever revealing a secret — no password to steal.

The thread across both lectures: security is about raising the cost for attackers — in time, computing power, or knowledge — until the attack isn't worth attempting.